

Git 핵심 요약 정리본

프로젝트 버전 관리 및 실무 관련 내용 모음

1. Git의 기본 구조와 로컬 작업 흐름

Git은 파일의 변경 이력을 누적하여 기록하는 버전 관리 도구이다. 파일이 언제, 어떻게, 누구에 의해 변경되었는지를 추적하여 문제 발생 시 안전하게 이전 상태로 되돌릴 수 있는 발판을 제공한다. 개발자의 로컬 작업 환경은 크게 3가지 영역으로 구분되며, 최종적으로 원격 저장소와 흐름이 연결된다.



- **Working Directory**: 사용자가 실제로 파일을 추가, 수정, 삭제하는 로컬 PC 상의 실제 작업 공간이다.
- **Staging Area**: 커밋을 실행하기 전, 이번 버전에 반영할 파일들만 선별하여 임시로 올려두는 완충 공간이다.
- **Repository (저장소)**: Git이 파일의 변경 이력을 스냅샷 형태로 영구히 기록하고 관리하는 프로젝트 폴더이다.

핵심 명령어

- **git init** : 현재 폴더를 Git 저장소로 초기화한다. 변경 이력을 기록할 숨김 폴더인 **.git** 이 생성된다.
- **git status** : 추적되지 않은 파일(Untracked), 수정된 파일, 스테이징된 파일의 상태를 조회한다. 작업 전후 필수 습관이다.
- **git add <파일명>** / **git add .** : 변경 사항을 Staging Area에 올린다. 점(**.**)은 현재 폴더 이하 모든 변경 사항을 뜻한다.
- **git commit -m "메시지"** : Staging Area에 올라간 파일들을 하나의 버전 기록으로 로컬 저장소에 영구히 저장한다.
- **git log --oneline** : 지금까지 로컬 저장소에 누적되어 온 커밋 기록들을 한 줄씩 간단하게 출력한다.

2. 브랜치(Branch)와 병합(Merge)

브랜치는 안전성이 확보된 메인 코드 흐름에서 독립되어, 기존 코드에 영향을 주지 않고 새로운 기능 개발이나 버그 수정을 개별적으로 실행할 수 있게 분리한 코드 작업 흐름이다. 작업 완료 후 안전성이 검증되면 다시 메인 코드와 결합한다.

[실무 이해를 돕기 위한 예시 시나리오 - 서빙 로봇 개발 팀]

매장 내 테이블로 주행이 가능한 로봇의 검증된 메인 코드가 `main` 브랜치에 보존되어 있다. 이 상태에서 두 명의 엔지니어가 각자 맡은 작업을 동시에 진행한다.

① 독립된 기능 브랜치 생성 및 이동

서로 작업 영역이 겹쳐 코드가 꼬이거나 메인 주행 로직이 고장 나는 것을 방지하기 위해 별도의 브랜치로 분가한다.

- A 엔지니어 (테이블 주문 연동): `git switch -c feature/order-ui`
- B 엔지니어 (장애물 회피 필터 개선): `git switch -c feature/lidar-filter`

② 독립된 작업 영역에서의 안전한 커밋

상대방의 소스코드 작업에 방해받지 않고, 자신이 구현한 기능 단위를 독립적으로 커밋하며 개발을 이어간다.

- A 엔지니어: 태블릿 연동 API 개발 완료 후 커밋 → `git commit -m "Add table order UI communication"`
- B 엔지니어: 돌발 장애물 인식 필터 값 튜닝 완료 후 커밋 → `git commit -m "Update lidar sensor noise filter"`

③ 기존 브랜치로의 안전한 병합 (Merge)

B 엔지니어의 장애물 회피 작업이 먼저 끝나 메인 프로그램 코드에 반영하고자 한다. **병합을 진행할 때는 반드시 흡수할 주체인 기존 브랜치로 이동한 후 합쳐야 한다.**

1. `git switch main` 검증용 기준이 되는 메인 브랜치로 다시 복귀한다.
2. `git merge feature/lidar-filter` 장애물 회피 브랜치의 변경 사항을 메인 브랜치에 가져와 결합한다.
3. `git branch -d feature/lidar-filter` 결합 처리가 완결되어 역할이 소멸한 기능 브랜치는 로컬에서 삭제한다.

이 흐름을 거치면 A 엔지니어가 주문 코드를 계속 작성하는 와중에도, 메인 코드는 B 엔지니어가 보완한 장애물 회피 로직이 안전하게 흡수되어 반영된다.

3. 원격 저장소(GitHub) 협업

- **Remote Repository**: GitHub, GitLab과 같은 외부 웹 서버 공간에 위치한 원격 저장소이며, 팀원 전체의 중앙 허브 역할을 수행한다.
- **origin**: 내 컴퓨터 로컬 저장소와 연동해 둔 긴 원격 저장소 URL 주소에 매핑된 기본 별명이다.
- **git clone <URL>**: 원격 저장소의 파일 소스, 전체 커밋 이력, 브랜치 정보를 내 컴퓨터로 최초에 고스란히 내려 받는다.
- **git push origin main**: 로컬 저장소에서 완수하여 기록한 커밋 버전들을 지정한 원격 저장소(**origin**)에 업로드한다.
- **git pull origin main**: 다른 팀원이 push하여 원격에 저장된 최신 소스코드 이력을 가져와서 내 현재 브랜치에 자동으로 병합한다.
- **git fetch origin**: 원격 저장소의 최신 변경 정보 이력만 조회하여 가져오고, 내 코드에 자동으로 합치지는 않는다. 변경 상태를 사전 파악할 때 안전하다.

4. 문제 해결 및 되돌리기 (Conflict & Reset)

충돌 (Conflict)

두 명 이상의 개발자가 같은 파일의 완전히 동일한 라인을 각자 수정하고 병합을 시도할 때, Git이 어느 쪽 코드를 채택 할지 스스로 결정하지 못해 멈추는 현상이다. (예: 설정 파일인 **nav2_params.yaml**의 특정 제어 속도 값을 한 명은 0.3, 다른 한 명은 0.5로 바꾸고 병합 시 발생)

- **해결 단계**: **git status**로 충돌 대상 파일을 탐색한 뒤, 해당 소스 파일을 연다. 파일 내에 수동으로 기입된 충돌 표시자(**<<<<<< HEAD**, **=====**, **>>>>>> 브랜치명**)를 확인하고 상호 협의된 최종 코드만 남긴 채 표시 기호들은 말끔히 지운다. 이후 **git add**를 수행하여 충돌 처리가 완료됨을 알리고 **git commit**으로 병합 커밋을 매듭짓는다.

복구 및 이력 관리 명령어

- **git restore <파일명>**: 수정 중이던 파일 내용을 전부 취소하고 가장 최신의 정상 커밋 상태로 소스코드를 강제 롤백시킨다. (커밋되지 않은 수정 사항은 소멸한다.)
- **git restore --staged <파일명>**: 실수로 **git add**를 실행하여 Staging Area에 등재해 버린 특정 파일만, 파일 내용은 보존한 채 스테이징 목록에서 다시 내려놓는다.
- **git revert <커밋ID>**: 특정 커밋의 오류를 취소하고 싶을 때, 기존의 이력을 삭제하는 하드 리셋 대신 **해당 커밋의 변경 사항만 거꾸로 되돌리는 새로운 취소 커밋을 만드는 방식**이다. 이미 원격(GitHub)에 push되어 공유된 이력을 파괴하지 않고 안전하게 복구할 수 있어 실무 협업 시 필수이다.

- **git stash / git stash pop** : 아직 커밋하기엔 불완전한 코드를 편집하던 중, 급한 업무 요청으로 브랜치를 변경해야 할 때 사용한다. 작업 상태를 임시 저장소에 잠시 격리 보관(**stash**)해 두었다가, 불일을 마치고 복귀하여 다시 소스코드 위로 꺼내와(**pop**) 연속해서 작업을 이어나간다.



5. 실무 프로젝트 관리 규칙

- **.gitignore 파일 운용**: 시스템 빌드 시 자동 생성되는 임시 파일, 컴파일 산출 대용량 폴더(예: ROS2 빌드 환경의 **build/** , **install/** , **log/**)나 개인 접속 비밀번호, 토큰 등이 노출될 위험이 있는 환경설정 파일(**.env**)을 Git 추적 대상에서 영구 제외하여 저장소를 쾌적하고 안전하게 보존한다.
- **Pull Request (PR) 프로세스**: 로컬에서 개발 및 테스트를 마친 기능 브랜치를 원격에 push한 뒤, **main** 에 곧바로 병합하지 않고 GitHub 플랫폼 상에서 동료 개발자들에게 변경 사항 리뷰를 공식 정중히 요청하는 절차이다. 코드 검토 및 리뷰를 거침으로써 메인 지점의 코드 품질을 유지하고 사소한 버그를 사전에 차단한다.
- **README.md 문서화**: 프로젝트를 처음 마주하는 팀원이나 후임자가 프로젝트의 성격, 개발 환경 구축법, 빌드 방식 및 실행 스크립트 명령어를 즉각 습득할 수 있도록 작성하는 마크다운 형식의 안내서이다.



6. 실무 엔지니어의 일일 표준 협업 루틴

순서	실행 명령어	실무 작업 내용 설명
1	git switch main git pull origin main	오전 출근 후, 원격 저장소에 병합된 다른 팀원들의 최신 메인 소스코드 상태를 내 로컬 저장소로 동기화한다.
2	git switch -c feature/task-name	오늘 나에게 할당된 명확한 작업 목표를 수행하기 위해 전용 기능 개발 독립 브랜치를 신설하고 이동한다.
3	(소스코드 변경 및 기능 테스트 수행)	지정된 브랜치 환경 내에서 코드를 구현하고 컴파일, 빌드 및 자체 디버깅 검증을 가동한다.
4	git status git add <변경된 파일명>	퇴근 전 작업이 완료되면 status 로 변경된 내역을 교차 검토하고, 스테이징 영역에 추가한다.
5	git commit -m "의미 있는 메시지"	무엇을 왜 변경했는지 타인이 명확히 알아볼 수 있도록 약속된 규칙에 따라 커밋 기록을 매듭짓는다.
6	git push origin feature/task-name	내 로컬 저장소 PC에 머물러 있던 온전한 기능 브랜치 커밋 내역을 GitHub 원격 저장소 서버에 업로드한다.
7	(GitHub 웹페이지에서 PR 생성)	GitHub 브라우저 창에서 메인 브랜치로 흡수를 요청하는 Pull Request 를 개설하고 리뷰를 공유한 뒤 퇴근한다.